# V5: A Complex-Native Algebraic Language Model with Phase-Preserving Nonlinearities and Parallel-Scan Selective Recurrence

Gowrav Vishwakarma
Independent Researcher, Ahmedabad, India
gowravvishwakarma@gmail.com
https://github.com/gowrav-vishwakarma/qllm2

**Abstract**

We present V5, a complex-native language modeling architecture built around a simple hypothesis: complex-valued parameters are only useful for language modeling if phase information is preserved through nonlinearities, gating, recurrence, and output projection. V5 combines complex embeddings and linear maps, phase-preserving activation via modReLU, complex gated units, multi-bank algebraic fusion, a selective complex diagonal state-space backbone implemented with parallel scan, and sparse phase-aware attention. The design is motivated by a falsifying result from an earlier model version (V4), where complex representations were repeatedly passed through real-valued nonlinear pathways and phase structure was not preserved. On completed internal TinyStories runs with the current `small-matched` configuration (28.7M parameters, 100k samples, 10 epochs), V5 reaches validation perplexity 11.77 with random initialization and 8.00 with orthogonal initialization. In separate seed-averaged initialization benchmarks, orthogonal initialization outperforms random by a large margin ($32.97 \pm 0.18$ vs. $47.86 \pm 0.19$ on a 5k-sample, 10-epoch setting). We frame these results as current preprint evidence for a mathematically cleaner complex-native redesign, not yet as proof of superiority over matched real-valued or transformer baselines, which remain future work.

## 1 Introduction

Most successful language models today are either attention-dominant transformers [10] or recurrence-centric state-space models (SSMs) [7, 6, 3]. Complex-valued neural computation has repeatedly shown benefits in domains with oscillatory or phase-rich structure, but adoption in language modeling remains limited and often inconclusive.

Our starting point is an adverse empirical signal from a controlled small-scale benchmark (TinyStories subset): a previous architecture (V4) trained much slower and converged to substantially worse perplexity than both a matched transformer and a matched diagonal recurrent baseline. Rather than post-hoc rationalization, we treat this as a null-hypothesis challenge.

**Core diagnosis.** Inspection of V4 revealed a structural mismatch: complex representations were introduced but repeatedly passed through real-valued nonlinear pathways that can corrupt or discard phase semantics. In other words, V4 paid complex-parameter costs without consistently preserving complex algebraic structure.

**V5 objective.** V5 is designed to test whether "complex-native end-to-end processing" can be made both principled and efficient:

- **Principled**: preserve phase through nonlinearities and gating;
- **Efficient**: use parallel scan for recurrent depth reduction and sparse attention for retrieval;
- **Falsifiable**: run matched ablations against real-valued counterparts.

**Contributions.**

1. A **complex-native LM stack** with explicit phase-preserving nonlinearity (modReLU) and complex gated units.
2. A **multi-bank algebraic fusion** module using learned per-bank phase rotations and diversity regularization.
3. A **selective complex diagonal SSM** implemented with parallel prefix scan [2], reducing recurrent sequential depth.
4. A **current evidence package** consisting of completed internal V5 runs, initialization benchmarks, and reproducibility metadata (commit hash and hardware).

# 2 Related Work

**Transformers and efficient attention.** Transformers established attention as the dominant sequence primitive [10]; many variants improve efficiency via sparse or local attention.

**State-space and recurrent alternatives.** S4 and successors showed long-range sequence modeling with structured linear recurrences [7]. Mamba introduced selective (input-dependent) recurrence with hardware-aware implementation [6]. Mamba-2 formalized state-space duality and improved systems efficiency [3]. Hybrid models combine recurrence with sparse attention [4, 8].

**Complex-valued and algebraic networks.** Complex-valued networks and activation design have been studied across signal-centric domains [9]. Unitary and complex recurrent models highlighted the importance of phase-preserving nonlinearities such as modReLU [1]. Recent theory revisits approximation properties in complex settings [5].

# 3 Method

## 3.1 Representation and primitives

V5 represents features as real-imaginary pairs $z \in \mathbb{R}^{d\times 2}$, corresponding to complex vectors. Core primitives are implemented in `v5/core/complex.py`:

- **ComplexLinear**: complex matrix multiplication via four real GEMMs;
- **ComplexNorm**: magnitude normalization with phase preservation;

- **ModReLU**: activation
$$\mathrm{modReLU}(z) = \mathrm{ReLU}(|z| + b)\frac{z}{|z| + \epsilon}, \tag{1}$$
preserving direction while thresholding magnitude;
- **ComplexGatedUnit (CGU)**: gate magnitude selects throughput, gate phase rotates transformed features before down-projection.

### 3.2 Multi-bank algebraic fusion

Each layer uses multiple AlgebraicBanks (pre-norm + CGU), then fuses them with:

1. per-bank projection to a common complex space,
2. learned unit-magnitude phase rotation per bank,
3. content-dependent real routing weights (softmax over magnitude-derived router logits),
4. weighted sum and output projection.

Bank specialization is encouraged via a diversity loss based on absolute complex cosine similarity between bank outputs.

### 3.3 Selective complex SSM with parallel scan

The recurrent core uses diagonal complex recurrence:
$$h_t = A_t \odot h_{t-1} + B_t, \quad y_t = Ch_t + D \odot x_t, \tag{2}$$
where $A_t$ is input-dependent through learned $\Delta t$ modulation. The scan operator
$$(a_1, b_1) \oplus (a_2, b_2) = (a_1 a_2, a_2 b_1 + b_2) \tag{3}$$
is associative, enabling Blelloch-style parallel prefix scan [2] for reduced sequential depth. Implementation: `v5/core/ssm.py`.

### 3.4 Sparse phase-aware attention

V5 inserts attention every $K$ layers (`attn_every_k`) with a sliding window. Scores use
$$s(q, k) = \mathrm{Re}(q \cdot \overline{k})/\sqrt{d_h}, \tag{4}$$
then apply real softmax and weighted complex value aggregation. Implementation: `v5/core/attention.py`.

### 3.5 Overall architecture

Given token IDs:

1. Complex embedding + norm;
2. Sequential bank stack with residual scaling;
3. Stacked complex SSM backbone with recurrent state carry;
4. Sparse phase attention blocks;
5. Tied complex LM head using the real part of a complex inner product with the embedding table.

See `v5/model.py`.

# 4 Experimental Protocol

## 4.1 Current reported experiments

The current preprint reports completed internal V5 experiments rather than a finished matched-baseline benchmark suite. The evidence package consists of:

- **Completed 100k-sample runs**: TinyStories, GPT-2 BPE, `small-matched`, sequence length 256, 10 epochs, comparing architecture revisions and initialization choices;
- **Initialization benchmarks**: smaller multi-seed studies on the `small` configuration, used to compare structured initialization strategies under controlled short-run settings;
- **Reproducibility metadata**: commit hash, GPU, batch size, and wall-clock time tracked per run in the experiment ledger.

## 4.2 Pending matched benchmark suite

- Dataset: TinyStories with a fixed split policy and exact token-count reporting;
- Tokenizer: GPT-2 BPE;
- Training schedule: AdamW with cosine decay and matched training budgets;
- Report: validation loss/perplexity, wall-clock time, hardware, and seed variance.

## 4.3 Planned baselines and ablations

- Transformer baseline matched by depth/width/parameter budget;
- Real-valued diagonal recurrent baseline (DiagRNN / linear SSM style);
- V5 ablations:

(a) remove banks,
(b) remove sparse attention,
(c) replace modReLU with real-part GELU surrogate,
(d) replace complex gates with real gates,
(e) real-valued counterpart with same macro-architecture.

## 4.4 Reproducibility checklist

We recommend reporting:

- exact commit hash, seed(s), and hardware;
- mixed-precision mode and runtime backend (CUDA/MPS/CPU);
- tokenized sample count and total train tokens;
- all hyperparameters from `v5/config.py`.

# 5 Results

## 5.1 External motivating comparison (prior model)

Table 1 summarizes the external controlled comparison that motivated V5 redesign.

Table 1: External controlled benchmark summary (provided feedback; not produced by this paper's final V5 runs).

| Model | Core Params | Best Val PPL | Time/Epoch |
|---|---|---|---|
| Transformer | ∼8M | 7.56 | 82s |
| DiagRNN / linear SSM | ∼9.5M | 9.18 | 512s |
| V4 | ∼11.9M | 17.05 | 1370s |

## 5.2 Completed internal V5 runs

Table 2: Current internal V5 run summary from the experiment ledger. Throughput is tracked in the ledger for reproducibility, but cross-GPU speed comparisons should not be treated as architecture-level evidence.

| Run | Params | Samples | Epochs | Init | Best Val PPL | GPU |
|---|---|---|---|---|---|---|
| v1-untied | 31.6M | 100k | 10 | random | 21.03 | A6000 |
| v2-tied | 18.7M | 100k | 10 | random | – | A6000 |
| v3-core-heavy | 28.7M | 100k | 1 | random | 66.64 | A6000 |
| v3-full | 28.7M | 100k | 10 | random | 11.77 | A6000 |
| v4-ortho | 28.7M | 100k | 10 | orthogonal | **8.00** | RTX 4090 |

**Interpretation.** The strongest completed architecture-level result in the current evidence package is the `v4-ortho` run, which uses the same `small-matched` architecture as `v3-full` but switches from random to orthogonal initialization and improves validation perplexity from 11.77 to 8.00.

## 5.3 Initialization results

Table 3: Initialization benchmark screening run (`small`, 1k samples, 5 epochs, 3 seeds).

| Strategy | Mean Val PPL | Notes |
|---|---|---|
| orthogonal | **168.27** | best overall |
| hadamard | 173.88 | close second |
| dft | 275.18 | viable but weaker |
| uniform | 289.08 | viable but weaker |
| random | 348.80 | baseline |

Table 4: Orthogonal vs. random initialization A/B benchmark (`small`, 5k samples, 10 epochs, 3 seeds).

| Strategy | Mean Val PPL | Std |
|---|---|---|
| orthogonal | **32.97** | 0.18 |
| random | 47.86 | 0.19 |

**Interpretation.** The initialization results are the most replicated finding in the current paper. Orthogonal initialization consistently outperforms random initialization across both the short screening benchmark and the longer 10-epoch A/B study, suggesting that initialization is a first-order factor for complex-valued training stability and quality in this architecture.

### 5.4 Reproducibility notes

The current experiment ledger records exact run metadata, including commit hash and hardware. For example, the strongest completed run (`v4-ortho`) corresponds to commit `e24acd2` on an RTX 4090, while earlier completed A6000 runs predate commit `bdf6b87` and are tracked as `pre-bdf6b87`. We recommend including this level of metadata in every reported row because GPU and batch-size differences affect runtime interpretation.

### 5.5 Ongoing full-dataset training

At the time of writing, a full TinyStories run with orthogonal initialization is in progress. Early partial results are promising (validation perplexity 6.27 after epoch 1 and 5.81 after epoch 2), but we do not treat these as finalized evidence in the main results tables until the run completes.

### 5.6 Future evaluation plan

The matched transformer and matched real-valued recurrent baselines remain the next critical revision milestone. Those comparisons are essential for deciding whether the complex-native redesign offers a real advantage over simpler alternatives, and they should be completed before making stronger competitiveness claims.

## 6 Discussion

V5 is intentionally framed as a testable hypothesis, not a completed claim of superiority. The key scientific question is not whether complex arithmetic is elegant, but whether preserving phase through nonlinear pathways yields measurable gains under controlled matched protocols.

**What would count as support?** Consistent improvements over matched real-valued baselines in either:

- perplexity at fixed training budget, or
- speed-quality tradeoff at fixed quality target.

One-off wins or mismatched comparisons should not be treated as evidence.

**What would falsify the hypothesis?** If matched V5 variants do not outperform or at least match real-valued counterparts after adequate tuning and equalized budgets, then the complex-native claim should be rejected for this regime.

## 7 Limitations

- The current paper reports internal V5 evidence, not yet a completed matched-baseline study.
- Some reported training runs use different GPUs and batch sizes, which limits speed comparisons.
- Parameter matching across complex and real models is subtle (feature width vs total real scalar count vs FLOPs).
- Small-data TinyStories outcomes may not transfer to larger corpora.
- Systems-level speed depends heavily on kernel implementation quality.

## 8 Ethical Statement

This work is fundamental model architecture research. Potential misuse follows standard language model risk channels (hallucination, misinformation, harmful text generation). We recommend standard safety filtering and evaluation procedures for downstream deployment.

## Code Availability

Project materials are publicly hosted at https://github.com/gowrav-vishwakarma/qllm2. However, the exact V5 implementation used for the experiments reported in this paper is still under active internal development and is not yet fully mirrored in the public repository. The current public repository reflects earlier project versions and supporting materials more completely than the latest V5 training code. We plan to release the stabilized V5 code, configurations, and experiment artifacts publicly once the implementation and reporting stack are cleaned up for reproducible release.

## Funding

This work was conducted without external funding.

## 9 Conclusion

We introduced V5, a complex-native language modeling architecture combining phase-preserving nonlinearities, complex gating, algebraic bank fusion, and parallel-scan selective recurrence. The current evidence supports two narrow conclusions: first, the V5 redesign is substantially more coherent and trainable than the earlier V4-style setup that motivated it; second, orthogonal initialization is a strong and replicated lever for this class of complex-valued models. The more ambitious question — whether complex-native language modeling outperforms matched real-valued alternatives — remains open and should be settled by the next round of matched baseline and ablation experiments.

**Code mapping.** Complex primitives: `v5/core/complex.py`; SSM: `v5/core/ssm.py`; banks/fusion: `v5/core/bank.py`; attention: `v5/core/attention.py`; model wiring: `v5/model.py`; protocol script: `v5/train.py`.

## References


[1] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *ICML*, 2016.

[2] Guy E. Blelloch. Prefix sums and their applications. Technical report, Carnegie Mellon University, 1990.

[3] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *ICML*, volume 235 of *PMLR*, pages 10041–10071, 2024.

[4] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

[5] Paul Geuchen, Thomas Jahn, and Hannes Matt. Universal approximation with complex-valued deep narrow neural networks. *Constructive Approximation*, 2025. doi: 10.1007/s00365-025-09713-8.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[7] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *ICLR*, 2022.

[8] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Schwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[9] Chiheb Trabelsi, Olexa Bilaniuk, Ying Zhang, Dmitriy Serdyuk, Sandeep Subramanian, João Felipe Santos, Soroush Mehri, Negar Rostamzadeh, Yoshua Bengio, and Christopher J. Pal. Deep complex networks. *ICLR*, 2018.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.


## A Appendix: Suggested Reporting Template

1. Hardware, runtime, and precision mode.

2. Full config dump.

3. Train/val token counts.
4. Wall-clock and peak memory.
5. Mean/std over seeds.